

Real-Time Examples of Polymorphism Principle in C#

In this article, I will discuss **Multiple Real-Time Examples of the Polymorphism Principle in C#**. Please read our [Polymorphism in C#](#) article, where we discussed the basic concepts of Polymorphism. At the end of this article, you will understand the following Real-time Examples using the Polymorphism Principle in C#.

1. [What is the Polymorphism Principle in C#?](#)
2. [Animals Sounds](#)
3. [Graphics Program](#)
4. [Payment Processing System](#)
5. [Managing a Fleet of Vehicles](#)
6. [Media Player](#)
7. [Notification System](#)
8. [Advantages and Disadvantages of Polymorphism Principle in C#](#)
9. [When to use Polymorphism in C#?](#)

What is the Polymorphism Principle in C#?

Polymorphism is a fundamental principle in object-oriented programming (OOP) that allows objects of different classes to be treated as objects of a common superclass. The term “polymorphism” originates from the Greek words “poly” (meaning many) and “morph” (meaning forms). Polymorphism enables you to invoke derived class methods through a base class reference in programming.

In C#, polymorphism is primarily achieved using inheritance, abstract classes, or interfaces combined with method overriding or overloading. There are two primary types of polymorphism in C#:

Compile-Time Polymorphism (Static Polymorphism):

- Achieved by method overloading or operator overloading.
- The method call is resolved at compile-time.

Run-Time Polymorphism (Dynamic Polymorphism):

- Achieved by method overriding.
- The method call is resolved at runtime based on the actual object type.
- In C#, this is implemented using the virtual keyword in the base class method and the override keyword in the derived class method.

Polymorphism is one of the four core principles of object-oriented programming (OOP). In C#, polymorphism allows you to invoke derived class methods through a base class reference.

Real-Time Example of Polymorphism Principle in C#: Animals Sounds

Here's a simple real-time example: consider animals making sounds. While every animal makes a sound, each animal's sound is distinct. You can leverage polymorphism to model this scenario. Let's walk through it:

- **Base Class: Animal**

- **Derived Classes: Dog, Cat**

Let us see how we can implement this example using the Polymorphism Principle in C#:

```
using System;
namespace PolymorphismExample
{
    // Base class
    public abstract class Animal
    {
        public abstract void MakeSound();
    }
    // Derived class
    public class Dog : Animal
    {
        public override void MakeSound()
        {
            Console.WriteLine("The dog barks.");
        }
    }
    // Another derived class
    public class Cat : Animal
    {
        public override void MakeSound()
        {
            Console.WriteLine("The cat meows.");
        }
    }
    //Testing Polymorphism Principle
    public class Program
    {
        static void Main(string[] args)
        {
            Animal myDog = new Dog();
            Animal myCat = new Cat();
            MakeAnimalSound(myDog); // Outputs: The dog barks.
            MakeAnimalSound(myCat); // Outputs: The cat meows.
            Console.Read();
        }
        // This function showcases polymorphism in action.
        // Even though it accepts a parameter of type 'Animal',
        // it's able to handle any derived type.
        static void MakeAnimalSound(Animal animal)
        {
            animal.MakeSound();
        }
    }
}
```

In this Example:

- We have an abstract Animal class with an abstract method, MakeSound().
- Derived classes (Dog and Cat) provide their own implementation of the MakeSound() method.
- In the Program class, even though we use the Animal type to hold references to the derived classes, we can still call the appropriate derived class's MakeSound() method. This is the essence of polymorphism.

This allows for flexibility and makes adding more animal types in the future easier without making major changes to existing code. If you were to add a new animal, say Bird, you'd need to create a Bird class derived from Animal and provide its own implementation for the MakeSound() method.

Real-Time Example of Polymorphism Principle in C#: Graphics Program

Let's understand polymorphism using another real-world example: a graphics program that can draw different shapes. Each shape can be drawn on a canvas, but the way each shape is drawn might differ. Here's how we can represent this using polymorphism in C#:

- **Base Class: Shape**
- **Derived Classes: Circle, Rectangle**

Let us see how we can implement this example using the Polymorphism Principle in C#:

```
using System;
namespace PolymorphismExample
{
    // Base class
    public abstract class Shape
    {
        public abstract void Draw();
    }
    // Derived class
    public class Circle : Shape
    {
        public override void Draw()
        {
            Console.WriteLine("Drawing a circle on the canvas.");
        }
    }
    // Another derived class
    public class Rectangle : Shape
    {
        public override void Draw()
        {
            Console.WriteLine("Drawing a rectangle on the canvas.");
        }
    }
    class Program
    {
        static void Main(string[] args)
        {
            Shape myCircle = new Circle();
```

```

        Shape myRectangle = new Rectangle();
        DrawShape(myCircle);           // Outputs: Drawing a circle on the canvas
        DrawShape(myRectangle);       // Outputs: Drawing a rectangle on the canvas
        Console.ReadKey();
    }
    // This function showcases polymorphism.
    // Even though it accepts a parameter of type 'Shape',
    // it's able to handle any shape derived from it.
    static void DrawShape(Shape shape)
    {
        shape.Draw();
    }
}

```

In this Example:

- The abstract base class Shape has an abstract method, Draw().
- The derived classes Circle and Rectangle provide their own implementations of the Draw() method.
- In the Program class, we can see polymorphism in action. We use a method DrawShape that can accept any object of type Shape (or derived from Shape) and call its Draw() method. Based on the actual object type (Circle or Rectangle), the appropriate Draw() method gets executed.

This way, if you needed to introduce new shapes in the future, like a Triangle or Polygon, you'd create new derived classes and implement their drawing logic without altering the existing base class or the DrawShape method.

Real-Time Example of Polymorphism Principle in C#: Payment Processing System

Let's understand polymorphism using another real-world example: Payment Processing System. Different payment methods, such as credit cards, bank transfers, and digital wallets, could have different processes to execute a payment. Here's how this can be represented with polymorphism:

- **Base Class: PaymentMethod**
- **Derived Classes: CreditCard, BankTransfer, DigitalWallet**

Let us see how we can implement this example using the Polymorphism Principle in C#:

```

using System;
namespace PaymentPolymorphismExample
{
    // Base class
    public abstract class PaymentMethod
    {
        public abstract void ExecutePayment(decimal amount);
    }
    // Derived class for credit card payment
    public class CreditCard : PaymentMethod
    {
        public override void ExecutePayment(decimal amount)
    }
}

```

```

    {
        Console.WriteLine($"Processing a credit card payment for ${amount}");
        // Here you'd have logic specific to credit card processing
    }
}
// Derived class for bank transfer
public class BankTransfer : PaymentMethod
{
    public override void ExecutePayment(decimal amount)
    {
        Console.WriteLine($"Processing a bank transfer for ${amount}.");
        // Logic specific to bank transfers would be here
    }
}
// Derived class for digital wallet payment
public class DigitalWallet : PaymentMethod
{
    public override void ExecutePayment(decimal amount)
    {
        Console.WriteLine($"Processing a digital wallet payment for ${amount}");
        // Logic specific to digital wallet payment would go here
    }
}
class Program
{
    static void Main(string[] args)
    {
        PaymentMethod creditCardPayment = new CreditCard();
        PaymentMethod bankTransferPayment = new BankTransfer();
        PaymentMethod digitalWalletPayment = new DigitalWallet();
        ProcessPayment(creditCardPayment, 100.00M);    // Outputs: Processing
credit card payment for $100.00.
        ProcessPayment(bankTransferPayment, 250.50M);    // Outputs: Processing
bank transfer for $250.50.
        ProcessPayment(digitalWalletPayment, 75.25M);    // Outputs: Processing
digital wallet payment for $75.25.
        Console.ReadKey();
    }
    // Demonstrating polymorphism.
    // This function can accept any payment method derived from PaymentMethod
    static void ProcessPayment(PaymentMethod paymentMethod, decimal amount)
    {
        paymentMethod.ExecutePayment(amount);
    }
}

```

In this Example:

- The PaymentMethod base class provides a contract with an abstract ExecutePayment method.

- Each derived class (e.g., CreditCard, BankTransfer, DigitalWallet) provides its own specific implementation for processing the payment.
- The ProcessPayment function in the Program class accepts any payment method (an object of a type derived from PaymentMethod) and processes the payment using polymorphism. This provides flexibility to introduce new payment methods in the future without changing the core payment processing logic.

When you run the above code, you will get the following output:

```
Processing a credit card payment for $100.00.
Processing a bank transfer for $250.50.
Processing a digital wallet payment for $75.25.
```

Real-Time Example of Polymorphism Principle in C#: Managing a Fleet of Vehicles

Let's understand polymorphism using another real-world example: Managing a Fleet of Vehicles. Each vehicle can be driven, but the driving experience and method might vary based on the type of vehicle. For example, you drive a car differently than you would pilot a boat.

- **Base Class: Vehicle**
- **Derived Classes: Car, Boat, Bicycle**

Let us see how we can implement this example using the Polymorphism Principle in C#:

```
using System;
namespace VehiclePolymorphismExample
{
    // Base class
    public abstract class Vehicle
    {
        public abstract void Drive();
    }
    // Derived class: Car
    public class Car : Vehicle
    {
        public override void Drive()
        {
            Console.WriteLine("Driving a car. Follow road signs!");
        }
    }
    // Derived class: Boat
    public class Boat : Vehicle
    {
        public override void Drive()
        {
            Console.WriteLine("Piloting a boat. Watch out for waves and other
        }
    }
    // Derived class: Bicycle
```

```

public class Bicycle : Vehicle
{
    public override void Drive()
    {
        Console.WriteLine("Riding a bicycle. Stay in the bike lane and wear a helmet!");
    }
}
class Program
{
    static void Main(string[] args)
    {
        Vehicle myCar = new Car();
        Vehicle myBoat = new Boat();
        Vehicle myBicycle = new Bicycle();
        OperateVehicle(myCar);           // Outputs: Driving a car. Follow road rules and waves and other vessels!
        OperateVehicle(myBoat);          // Outputs: Piloting a boat. Watch for waves and other vessels!
        OperateVehicle(myBicycle);       // Outputs: Riding a bicycle. Stay in the bike lane and wear a helmet!
        Console.ReadKey();
    }
    // Function showcasing polymorphism.
    // Accepts any vehicle type and drives it.
    static void OperateVehicle(Vehicle vehicle)
    {
        vehicle.Drive();
    }
}

```

In this Example:

- The Vehicle is the base class with an abstract method, Drive.
- The derived classes Car, Boat, and Bicycle each provide their own unique implementations of the Drive method.
- The OperateVehicle function in the Program class can accept any object of type Vehicle (or derived from Vehicle) and invokes the appropriate Drive method due to polymorphism.

Should you need to introduce a new type of vehicle, such as a “Helicopter,” you’d create a new derived class from the Vehicle and implement a specific way to pilot it. No changes to the OperateVehicle method would be necessary.

Real-Time Example of Polymorphism Principle in C#: Media Player

Let’s understand polymorphism using another real-world example: a media player application that can play various media files, such as audio and video. Here’s how this can be structured:

- **Base Class: MediaFile**
- **Derived Classes: AudioFile, VideoFile**

Let us see how we can implement this example using the Polymorphism Principle in C#:

```
using System;
namespace MediaPlayerPolymorphismExample
{
    // Base class
    public abstract class MediaFile
    {
        public string FileName { get; set; }
        public MediaFile(string fileName)
        {
            FileName = fileName;
        }
        public abstract void Play();
    }
    // Derived class: AudioFile
    public class AudioFile : MediaFile
    {
        public AudioFile(string fileName) : base(fileName) { }
        public override void Play()
        {
            Console.WriteLine($"Playing audio file: {FileName}.");
        }
    }
    // Derived class: VideoFile
    public class VideoFile : MediaFile
    {
        public VideoFile(string fileName) : base(fileName) { }
        public override void Play()
        {
            Console.WriteLine($"Playing video file: {FileName}.");
        }
    }
    class Program
    {
        static void Main(string[] args)
        {
            MediaFile mySong = new AudioFile("song.mp3");
            MediaFile myMovie = new VideoFile("movie.mp4");
            PlayMedia(mySong);    // Outputs: Playing audio file: song.mp3.
            PlayMedia(myMovie);   // Outputs: Playing video file: movie.mp4.
            Console.ReadKey();
        }
        // Function illustrating polymorphism.
        // It can accept any media type derived from MediaFile and play it.
        static void PlayMedia(MediaFile media)
        {
            media.Play();
        }
    }
}
```

```
}
```

In this Example:

- The base class `MediaFile` provides an abstract `Play` method. It also holds a common property, `FileName`, to identify each media file.
- The derived classes `AudioFile` and `VideoFile` each offer their own specific implementations of the `Play` method.
- The `PlayMedia` function in the `Program` class can handle any object type derived from `MediaFile` and invoke the appropriate `Play` method, showcasing polymorphism in action.

If, in the future, the application needs to support other media formats, like `ImageFile` for slideshows, you can easily extend the system by creating a new derived class from `MediaFile` without having to modify existing player logic.

Real-Time Example of Polymorphism Principle in C#: Notification System

Let's understand polymorphism using another real-world example: a notification system in which different notification methods (e.g., Email, SMS, Push Notification) are utilized. Here's the setup:

- **Base Class:** `Notification`
- **Derived Classes:** `EmailNotification`, `SmsNotification`, `PushNotification`

Let us see how we can implement this example using the Polymorphism Principle in C#:

```
using System;
namespace NotificationSystem
{
    // Base class
    public abstract class Notification
    {
        public string Recipient { get; set; }
        public string Message { get; set; }
        public Notification(string recipient, string message)
        {
            Recipient = recipient;
            Message = message;
        }
        public abstract void Send();
    }
    // Derived class: EmailNotification
    public class EmailNotification : Notification
    {
        public string Subject { get; set; }
        public EmailNotification(string recipient, string subject, string message)
            : base(recipient, message)
        {
            Subject = subject;
        }
        public override void Send()
```

```

        {
            Console.WriteLine($"Sending Email to {Recipient} with subject: {Subject} and message: {Message}");
        }
    }
    // Derived class: SmsNotification
    public class SmsNotification : Notification
    {
        public SmsNotification(string phoneNumber, string message)
            : base(phoneNumber, message) { }
        public override void Send()
        {
            Console.WriteLine($"Sending SMS to {Recipient} with message: {Message}");
        }
    }
    // Derived class: PushNotification
    public class PushNotification : Notification
    {
        public PushNotification(string device, string message)
            : base(device, message) { }
        public override void Send()
        {
            Console.WriteLine($"Sending Push Notification to device {Recipient} with message: {Message}");
        }
    }
    class Program
    {
        static void Main(string[] args)
        {
            Notification email = new EmailNotification("user@example.com", "Hello! This is a notification email.");
            Notification sms = new SmsNotification("123-456-7890", "This is an SMS notification.");
            Notification push = new PushNotification("Device123", "This is a push notification.");
            SendNotification(email); // Outputs: Sending Email to user@example.com with subject: Hello! and message: This is a notification email.
            SendNotification(sms); // Outputs: Sending SMS to 123-456-7890 with message: This is an SMS notification.
            SendNotification(push); // Outputs: Sending Push Notification to Device123 with message: This is a push notification.
            Console.ReadKey();
        }
        // Function illustrating polymorphism.
        // It can send any type of notification derived from the Notification
        static void SendNotification(Notification notification)
        {
            notification.Send();
        }
    }

```

}

In this Example:

- The base class Notification has properties like Recipient and Message and an abstract method Send.
- The derived classes (EmailNotification, SmsNotification, PushNotification) each provide specific implementations for the Send method.
- The SendNotification method in the Program class accepts any notification object derived from the base Notification class and sends it using polymorphism.

This approach allows for easy scalability. If, in the future, you wanted to add a new type of notification, say a "SlackNotification," you'd derive it from the base Notification class and provide a specific Send implementation.

Advantages and Disadvantages of Polymorphism Principle in C#

Polymorphism is one of the four core principles of object-oriented programming (OOP), and like any programming concept, it comes with its own advantages and disadvantages.

Advantages of Polymorphism Principle in C#:

- **Flexibility and Extensibility:** You can introduce new classes or methods that implement or override existing functionality without changing existing code.
- **Code Reusability:** Common behaviors can be defined in a superclass and inherited in multiple subclasses, eliminating the need for redundant code.
- **Maintainability:** Polymorphism, when combined with good design practices, can simplify the codebase, making it easier to understand, modify, and maintain.
- **Unified Interface:** It provides a single interface to entities of different types. This means you can use a unified way to interact with objects regardless of their specific classes. For instance, you might have a method that works on a Shape object, and it would work for any derived class (Circle, Rectangle, etc.) without needing a separate method for each shape.
- **Dynamic Dispatch:** With runtime polymorphism, you can decide at runtime which method or property to call, enabling dynamic behaviors based on the actual object and not just its reference type.
- **Decoupling:** Polymorphism can lead to decoupled systems where changes in one part (e.g., a subclass implementation) might not affect others using its polymorphic interface.

Disadvantages of Polymorphism Principle in C#:

- **Complexity:** Introducing polymorphism can make the system more complex, especially for those unfamiliar with the concept. There can be layers of inheritance and overridden methods to navigate through.
- **Overhead:** Dynamic dispatch (a feature of runtime polymorphism) can introduce a slight overhead since method calls are resolved at runtime. This is typically not an issue for most applications, but it might be a consideration in high-performance scenarios.
- **Debugging Difficulties:** Tracing through polymorphic method calls can be more challenging since executing the actual method depends on the runtime object's type. It may not always be evident which method is being called, especially in deep inheritance hierarchies.
- **Design Constraints:** It can sometimes force a design to fit the "is-a" relationship required by inheritance, even when it might not fit best.

- **Potential Misuse:** Inappropriately designed inheritance hierarchies or excessive overriding can lead to a system that's hard to understand or modify. A classic pitfall is the “diamond problem” faced in some languages, although C# avoids this issue by not allowing multiple inheritance for classes.
- **Tight Coupling (if misused):** If polymorphism and inheritance are misused, it can introduce tight coupling between parent and child classes, making the system harder to change.

When to use Polymorphism in C#?

Here are some common situations where employing polymorphism in C# would be beneficial:

- **Common Interface with Different Implementations:** Polymorphism is ideal if you have several classes that should provide the same set of operations but with different implementations. For example, a set of Shape classes (Circle, Rectangle, Triangle, etc.) might all have a Draw method, but the method's implementation will vary based on the specific shape.
- **Reusable Code:** Polymorphism can be helpful when you want to write generic code that can work with objects of different types. For instance, a Render(Shape shape) function can render any shape passed to it, regardless of the specific derived type of the shape.
- **Extensibility:** Polymorphism is a good choice if you're building a system where you expect to add new types or behaviors in the future without changing existing code (following the open/closed principle). For instance, you might add new shapes or transportation vehicles to a system without modifying the existing rendering or transportation logic.
- **Framework and Library Development:** When developing libraries or frameworks where you don't know all the specific implementations users might require, you can provide base classes or interfaces. Users can then create derived classes tailored to their needs. Entity Framework, ASP.NET Core middleware, and other components in .NET use this approach.
- **Decoupling and Dependency Injection:** Polymorphism helps decouple your system components. With Dependency Injection (DI) frameworks like the one in ASP.NET Core, you can inject specific implementations of interfaces (polymorphic behavior) to achieve decoupling and enhance testability.
- **Strategy Pattern:** You can use polymorphism to define a family of algorithms or strategies and make them interchangeable. Each algorithm can be encapsulated in its own class with a common interface. At runtime, you can select and use the desired strategy.
- **Factory Pattern:** When creating objects where the exact type of the object isn't known until runtime, factories utilizing polymorphism can produce the required object type based on runtime data.
- **Unit Testing and Mocking:** Polymorphism aids in unit testing. You can create mock objects with the same interface as your actual objects. These mock objects can simulate the behavior of real objects, making unit tests more isolated and reliable.

However, while there are numerous scenarios where polymorphism is advantageous, avoiding its unnecessary use is essential. Over-engineering, creating complex inheritance hierarchies, or introducing polymorphism without a clear purpose can lead to a harder-to-understand, maintain, and modify system. Always aim for the right balance and ensure that the use of polymorphism aligns with your system's goals and requirements.

In the next article, I will discuss the [Multiple Real-time Examples of Interface in C#](#). In this article, I explain **Real-Time Examples of the Polymorphism Principle in C#**. I hope you enjoy this Polymorphism Principle in Real-Time Examples using the C# article.